Implementation of a Complete Single-Cycle RISC-V Processor in Verilog HDL

Muhammad Zoraiz Husnain
Department of Computer Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Reg number: 2024498

Farzeen Fatima
Department of Computer Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Reg number: 2024171

Zahra Aliabbas
Department of Computer Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
Reg number: 2024668

***Abstract*—This paper presents the design and implementation of a 32-bit single-cycle RISC-V processor using Verilog Hardware Description Language (HDL). The processor supports a representative subset of the RV32I base integer instruction set, including arithmetic, logical, memory access, branch, and jump instructions. The design strictly follows the classical single-cycle datapath architecture, ensuring that the entirety of an instruction's execution occurs within a single clock period. Verification is achieved using a comprehensive Register Transfer Level (RTL) simulation testbench that analyzes internal states and data memory interactions. The processor achieves a Cycles Per Instruction (CPI) of exactly 1, with overall performance and maximal operating frequency constrained by the critical path associated with the load-word instruction.**

***Index Terms*—RISC-V, Verilog HDL, Single-Cycle Processor, Microarchitecture, Datapath, Control Unit, VLSI Design**

I. Introduction

The Reduced Instruction Set Computer-Five (RISC-V) architecture has emerged as a transformative, open-standard Instruction Set Architecture (ISA). Unlike proprietary, encumbered ISAs such as x86 or ARM that dominate the commercial microprocessor landscape and require expensive licensing agreements, RISC-V offers a completely royalty-free and highly modular framework. This inherent transparency enables unrestricted exploration into hardware design, making it a ubiquitous standard in advanced academic research, educational environments, and custom embedded system deployments.

The foundational design philosophy of the RISC-V RV32I base integer instruction set is stark simplicity combined with extensibility. By restricting the base ISA to a minimal, highly optimized subset of operations, the architecture provides a Turing-complete foundation that avoids the legacy bloat typical of Complex Instruction Set Computers (CISC).

In a single-cycle implementation, the processing paradigm dictates that every instruction must execute sequentially and completely within one clock period. Consequently, the five fundamental stages of instruction processing—Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Register Write-Back (WB)—are cascaded purely as combinational logic bounded by state registers (the Program Counter and the Register File). While this dramatically simplifies control logic by entirely eliminating pipeline hazards, data dependencies, and the need for complex branch prediction algorithms, it introduces severe temporal constraints. The clock period must be elongated to accommodate the slowest possible instruction.

This project outlines the exhaustive hardware implementation of a single-cycle RISC-V RV32I processor in Verilog HDL. The subsequent sections deconstruct the system architecture, dissect the individual datapath hardware modules, analyze the combinatorial control unit logic, and evaluate the verification methodology.

II. System Architecture

The top-level architecture of the processor is categorically segregated into two interacting domains: the Datapath and the Control Unit. The datapath comprises the physical interconnects, computational units, and storage matrices required to route and process binary data. Conversely, the Control Unit acts as the deterministic coordinator, decoding the incoming instruction fields and asserting the appropriate multiplexer selection lines to steer data appropriately.

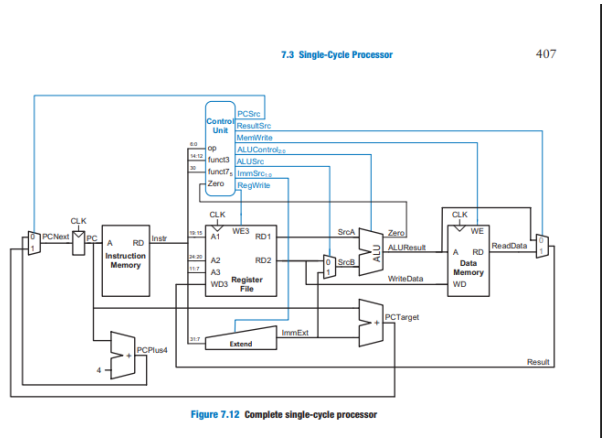

Fig. 1. Single-Cycle RISC-V Datapath Overview

The continuous execution flow within a single clock cycle is strictly organized as follows:

1) **Instruction Fetch:** The 32-bit Program Counter (PC) outputs the current instruction address. This queries

the Instruction Memory, yielding the compiled 32-bit instruction word.

2) **Instruction Decode:** The instruction is fragmented into specific sub-fields (Opcode, `rs1`, `rs2`, `rd`, `funct3`, `funct7`). The Register File simultaneously accesses the data mapped to `rs1` and `rs2`, while the Control Unit translates the Opcode into active control signals.
3) **Execution:** The Arithmetic Logic Unit (ALU) ingests operands either directly from the Register File or from the Immediate Generation unit. It performs the mathematical synthesis and generates condition flags.
4) **Memory Access:** The computed output from the ALU serves as an effective physical address to interface with the Data Memory block for load and store operations.
5) **Write-Back:** The terminal stage routes the final resultant data back into the destination register (`rd`) within the Register File on the subsequent active clock edge.

To orchestrate this flow, a top-level hardware module binds the Control Unit, Datapath, and Data Memory together, routing interconnect wires such as the `ALUResult`, memory read/write buses, and the various ALU status flags.

III. Datapath Design

The datapath dictates the maximal throughput and physical footprint of the synthesized processor. Each module relies on robust digital logic design principles to operate correctly within a single clock span.

A. Program Counter (PC) and Multiplexer Logic

The Program Counter is an edge-triggered 32-bit register equipped with an asynchronous active-low reset. In nominal sequential operation, the PC is augmented by 4 bytes (the fixed width of an RV32I instruction) via a dedicated, hardwired arithmetic adder.

When a control flow instruction modifies the execution path, the next-state logic routes a computed offset address into the PC input. This is achieved using a 3-to-1 multiplexer governed by a `PCSrc` control vector. The multiplexer selects between sequential execution ($PC + 4$), branch/jump targets computed by adding the sign-extended immediate to the current PC, and the `ALUResult` (used specifically for the `JALR` instruction to jump to a register-relative address).

B. Register File Structure

The RV32I standard necessitates 32 general-purpose registers, each natively 32 bits wide. To support true single-cycle execution, the physical memory module requires dual asynchronous read ports to fetch operands seamlessly without clock delay, and a single synchronous write port bounded to the positive clock edge.

A strict architectural mandate of RISC-V is that register `x0` is absolutely immutable and hardwired to zero. In our structural Verilog modeling, any attempt to read from address `5'b00000` inherently returns a 32-bit zero vector, and write enable signals targeting this address are effectively ignored by the hardware. This optimization is heavily utilized by the compiler to synthesize pseudo-instructions, such as utilizing an addition with `x0` to implement a register-to-register move.

C. Arithmetic Logic Unit (ALU)

The ALU serves as the computational nucleus of the execution stage. It operates as purely combinational logic, directed by a 4-bit `ALUControl` signal. It computes fundamental arithmetic (Addition, Subtraction), bitwise logic (AND, OR, XOR), and complex shifting maneuvers (Shift Left Logical, Shift Right Logical, Shift Right Arithmetic). Furthermore, the ALU computes a Set Less Than (`SLT`) evaluation utilizing full signed two's complement comparison.

To execute subtraction without a dedicated sub-circuit, the ALU performs two's complement addition. If the control signal indicates a subtraction, the `B` operand is inverted (using a bitwise NOT gate array), and a `1` is injected into the carry-in of the main adder.

In tandem with the primary 32-bit mathematical result, the ALU exports an array of boolean condition flags essential for control-flow branching:

- **Zero (Z):** Asserted high exclusively when the 32-bit result vector is composed entirely of logical zeros.
- **Negative (N):** Mirrors the most significant bit (MSB) of the calculation result.
- **Overflow (V):** Asserted when an arithmetic boundary violation occurs during signed computations. Mathematically, it is true if the operands share the same sign, but the resultant sum possesses the opposite sign.
- **Carry (C):** Asserted during unsigned addition if the result exceeds 32 bits.

Additional diagnostic flags, including Parity (P), Borrow, Sign (S), and Half-Carry (H), are also generated concurrently to assist in advanced debugging and potential extensions to the ISA.

D. Immediate Extension Logic

Because RISC-V instructions are strictly 32 bits wide, large constant values (immediates) must be compressed and spliced across various fields within the instruction encoding. The Immediate Generation unit unpacks these fragmented bits and sign-extends them into full 32-bit values prior to ALU execution.

- **I-Type:** Extracts bits [31:20] and sign-extends the 12-bit value.
- **S-Type:** Concatenates bits [31:25] and [11:7] to form the store offset.
- **B-Type:** Reconstructs a 13-bit offset by concatenating bit 31, bit 7, bits [30:25], and bits [11:8], padding the least significant bit with a zero since instructions must be half-word aligned.
- **J-Type:** Extracts a 21-bit jump offset from bits [31:12] using a complex reshuffling matrix.

E. Memory Subsystem

To prevent catastrophic structural hazards within a single clock span, the microarchitecture adopts a strict Harvard memory paradigm at the highest physical level, logically segregating the Instruction Memory from the Data Memory. Instruction memory is implemented as asynchronous read-only memory, loaded at simulation initialization from a pre-compiled hex file. The Data Memory supports byte-addressable, word-aligned storage operations, mapped directly to the ALU's output to calculate dynamic target addresses during load/store commands.

IV. Control Unit Synthesis

The Control Unit is a vast combinational logic cloud that orchestrates the active state of the datapath multiplexers. We modularized this entity into a Main Decoder and an ALU Decoder to optimize synthesis mapping and improve human readability.

A. Main Decoder Configuration

The Main Decoder ingests the 7-bit Opcode field (Instruction bits [6:0]). It determines the macro-instruction archetype and outputs the macroscopic control vectors to the datapath.

TABLE I
Main Decoder Combinational Truth Table

Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc
0110011 (R-type)	1	xx	0	0	00
0010011 (I-type)	1	00	1	0	00
0000011 (lw)	1	00	1	0	01
0100011 (sw)	0	01	1	1	00
1100011 (beq)	0	10	0	0	00
1101111 (jal)	1	11	0	0	10

The `ALUSrc` signal decides whether the ALU's second operand comes from the register file (for R-type instructions) or the sign-extended immediate (for loads, stores, and I-type arithmetic). The `ResultSrc` vector dictates the final write-back multiplexer, selecting between the ALU result (`00`), the Data Memory read output (`01`), or the PC + 4 return address (`10`) reserved specifically for jumping subroutine linkages (`JAL`).

B. ALU Decoder and Branch Logic

While the Main Decoder dictates the flow of data, the ALU Decoder defines the exact mathematical operation. It processes a 2-bit `ALUOp` signal from the main decoder alongside the instruction's `funct3` (bits [14:12]) and bit 30 (`funct7` modifier). For instance, when an R-type arithmetic instruction is decoded, bit 30 acts as the critical toggle between a standard Addition and a Subtraction, as well as toggling between Logical and Arithmetic right shifts.

Branch resolution is cleanly decoupled from the main ALU operation to expedite timing. A dedicated Branch Logic gate array processes the exported ALU flags and the specific branch operation defined by `funct3`.

- **BEQ:** Branch taken if the Zero flag is 1 (`funct3 == 3'b000`).
- **BNE:** Branch taken if the Zero flag is 0 (`funct3 == 3'b001`).
- **BLT/BGE:** Evaluated by taking the exclusive-OR of the Negative and Overflow flags (`N ^ V`).

V. Instruction Set Support

To achieve practical utility, the processor implements a robust subset of the RV32I specification. The processor parses assembly code mapped into the following distinct functional formats:

- **R-type (Register):** Operations involving three registers natively supporting `ADD`, `SUB`, `SLL`, `SLT`, `XOR`, `SRL`, `SRA`, `OR`, and `AND`.
- **I-type (Immediate):** Operations involving a 12-bit immediate, utilized heavily for loop indices (`ADDI`, `ANDI`, `ORI`, `SLTI`).
- **S-type (Store):** The `SW` instruction computes an effective address and writes a 32-bit register payload into memory.
- **B-type (Branch):** PC-relative conditional jumps (`BEQ`, `BNE`, `BLT`, `BGE`) that form the backbone of conditional loops.
- **Jump & Upper Immediates:** `JAL` and `JALR` accommodate subroutine jumping while securely storing the return vector. `LUI` and `AUIPC` construct large 32-bit constants dynamically.

VI. Simulation and Verification

Hardware verification is essential to ensure that the logical implementation correctly mirrors the ISA specification. A comprehensive testbench was developed to iteratively step the processor through all instruction types.

A. Control Unit Waveform Analysis

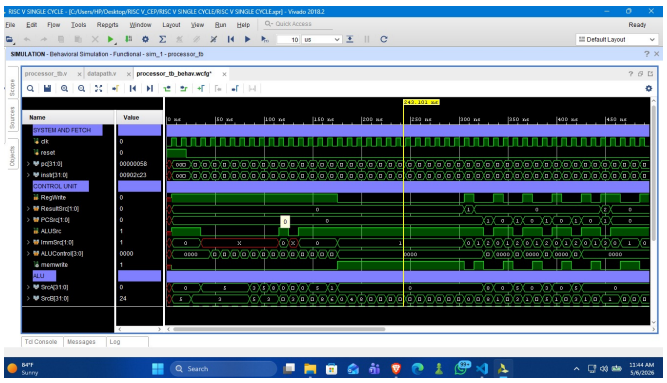

Fig. 2. Control Unit Signals Trace

Simulation waveforms were analyzed extensively to validate macro-state behavior. For example, during the execution of a store instruction (`SW`) at program counter 0×58:

- The `MemWrite` signal is asserted high, enabling the data memory block for writing.
- The `RegWrite` signal is correctly held low, protecting the register file from accidental corruption.

- The `ALUSrc` multiplexer line is set to 1, ensuring the immediate offset is routed into the ALU rather than a secondary register value.

These trace logs verify the flawless translation of opcode bit-patterns into physical, physical control lines.

B. Arithmetic and Subsystem Verification

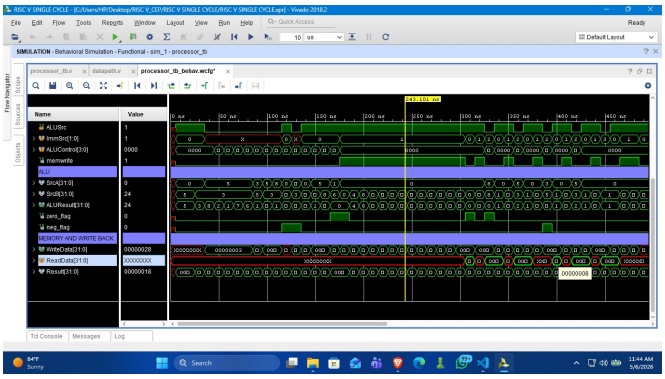

Fig. 3. ALU and Memory Data Bus Verification

The automated Verilog testbench executes an array of localized combinatorial checks across the ALU boundary using manual PASS/FAIL assertions tied to the clock cycle.

- **Mathematical Logic:** Verified that standard operations yield precise integers (e.g., `ADD`: $5+3=8$; `SUB`: $4-2=2$).
- **Shift Integrity:** Validated that barrel shifting algorithms push bits without truncation errors (e.g., `SLL` converting binary 5 left by 3 units correctly outputs 40).
- **Memory Access:** The sequence of a `SW` followed by a `LW` command successfully wrote to a computed address and immediately fetched the stored dataset securely without data loss.

VII. Performance Analysis and Constraints

The cardinal metric of a single-cycle datapath is its execution throughput. By design, the architecture strictly maintains a Cycles Per Instruction (CPI) of exactly 1.0. This architectural simplicity provides unparalleled predictability for real-time systems.

However, while a CPI of 1 implies high efficiency per clock cycle, the processor's maximum physical clock frequency (F_{max}) is severely bottlenecked. The clock period (T_c) must be greater than or equal to the time taken by the longest possible path through the datapath combinational logic.

For the RISC-V ISA, the critical path universally occurs during the load-word (`lw`) instruction. This instruction spans the absolute longest physical route through the circuitry, mathematically defined as:

$$T_c \geq t_{pc\_q} + t_{imem} + t_{decode} + t_{reg\_read} + t_{alu} + t_{dmem} + t_{mux} \quad (1)$$

Because both the instruction memory fetch and the data memory access impose massive, localized propagation delays, summing them sequentially alongside the ALU computation drastically elongates the minimum required clock period. Consequently, while the processor calculates instructions correctly, it is statically locked to a low maximum theoretical frequency compared to modern super-scalar designs.

VIII. Conclusion

This paper comprehensively details the design, implementation, and functional verification of a 32-bit single-cycle RISC-V microarchitecture. By translating standard architectural specifications into structural Verilog components, we achieved full functional compliance with a robust subset of the RV32I base integer instruction set. The rigorous RTL simulation validated that the combinational control matrices accurately command the underlying datapath without instruction collisions, signal degradation, or state corruption. This core serves as an optimized, structurally sound baseline for advanced computer architecture exploration and educational hardware modeling.

IX. Future Work

To circumvent the physical frequency limitations modeled in the performance analysis, the immediate future trajectory for this project involves deep architectural refactoring. Specifically, future enhancements will aim to:

- **Implement Pipelining:** Segment the datapath into the standard 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) to vastly reduce the critical path length, allowing a massive increase in clock frequency at the expense of an increased CPI.
- **Hazard Resolution:** Design dynamic forwarding logic matrices and hazard stalling networks (bubble insertion) to manage the data dependencies and control hazards inherent to pipelined execution.
- **Cache Integration:** Deprecate the idealized, immediate-response single-cycle memory modules in favor of realistic L1 Instruction and Data Caches, which will require advanced state machines to handle read/write latency misses dynamically.

References

[1] A. Waterman and K. Asanović, "The RISC-V Instruction Set Manual," RISC-V Foundation, 2019.

[2] D. Patterson and J. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, RISC-V Edition, Morgan Kaufmann, 2021.

[3] S. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2022.